

Read the Code

Why the harness is not enough and why software factories fail

Talk by Dex Horthy (HumanLayer) | Notes from 3 October 2026

Source: <https://www.youtube.com/watch?v=Ib5GBkD555M>

In one line: AI coding agents are great at making tests pass and bad at keeping a codebase healthy. No amount of clever tooling around the model fixes that, so you still have to read the code. The way to stay fast is to plan carefully up front so review is easy.

Part 1: Summary of the talk

The argument

1. **The popular story.** Humans are the bottleneck, code is free, so spend more tokens and stop reading the code. The extreme is the "lights-off" software factory (StrongDM), where nobody reviews anything.
2. **The cracks.** Outages caused by coding agents, more incidents and bugs per developer, worse PR review, and many PRs merged with no review at all.
3. **His claim.** This is not a skill issue. It is a model training issue, which is why "the harness is not enough."
4. **Why models are like this.** Coding models are trained with reinforcement learning on short tasks with a pass/fail reward: did the tests pass without breaking anything else? Nothing penalizes poor design, so you get needless try/catches and type casts just to get to green.
5. **Why it is hard to fix.** Bad architecture costs you months later, and that signal cannot easily be fed back into training. Newer benchmarks (SWE-Marathon, DeepSWE, Frontier Code) are improving, but a model judging code quality only goes so far. If it knew what good code looked like, it would have written it.
6. **His evidence.** HumanLayer went fully lights-off in July 2025. Eventually they hit a bug the agent could not solve, in a codebase nobody had read for three months, while the site was down.

Key ideas worth remembering

- **Not about vibe coding.** The talk is about hard problems in complex codebases, not side projects a dozen people will run.
- **Brownfield arrives early.** It used to mean a 10-year-old Java system. Agents now start to struggle after roughly 3 to 6 months.
- **The core shortcoming.** Models cannot maintain or improve codebase quality over time without a decent amount of human steering.
- **Shotgun surgery.** The Martin Fowler code smell where a change in one place forces changes, or breaks things, in many others.

- **Uneven progress.** Models have become far better at one-off problems, but not noticeably better at keeping a codebase maintainable.

What to do instead

Turn the lights back on and keep code review, but move the effort to the front. Small tasks still go straight to the agent. For bigger ones:

- **Product review:** what problem, what behaviour, mockups.
- **Architecture:** component contracts, data models, constraints.
- **Program design:** types, method signatures, layout, call stacks. This is the step he says people skip.
- **Vertical slices:** the order of implementation and how you check each phase.

About 30 minutes of this saves hours of review, which makes reading every line feasible. In his words, you do not have too many PRs, you have "too many bad PRs."

Two caveats

- **Call graphs are a planning tool here.** You map how functions will call each other before the agent writes code, rather than using them for debugging or optimization.
- **Weigh the source.** The talk closes with a pitch for HumanLayer, and he admits he cannot prove models have not improved at maintainability, because no good benchmark exists.

Part 2: Reflection by Keerthi

The observation

The time it takes for a codebase to become legacy seems to be shrinking with AI-driven coding.

Developers were always incentivized to ship rather than think about architecture, quality and security. But in the past they had no option but to think about those things at least a little in order to start development. With AI, that judgement step is almost gone, which is much scarier.

Sharpening the idea

- **Legacy means "nobody understands it."** That used to take years of turnover and drift. With agents, code can be legacy the day it is merged, because nobody ever held it in their head in the first place.
- **Slowness was an accidental design review.** You could not type out a function without deciding where it lived, what it took and what it touched. Nobody chose to think about architecture; the work forced a minimum amount of it.
- **Judgement is unbundled, not gone.** AI removed the friction, so the thinking went from unavoidable to optional. Since incentives always pointed at shipping, optional mostly means skipped.

- **Planning rebuilds the forcing function.** The upfront stages in the talk deliberately put back what typing used to provide for free.
- **The longer-term worry.** That forced thinking was also how developers built judgement. If juniors never go through it, it is unclear where the next generation of capable reviewers comes from.